

GeoPulse

Spatial Index Benchmarking for NYC Collision Data

Team Name:	GeoPulse Solo Team
Team Member:	Guilherme Armin Da Silva Anton
UFID:	2641-9801
GitHub Username:	arminanton
Repository:	https://github.com/arminanton/project-2
Video Demo:	https://youtu.be/REPLACE_WITH_FINAL_VIDEO
Language:	C++14
Submission:	Final group deliverable report

Summary. GeoPulse compares Linear Scan, KD-tree, and Quadtree spatial search over the NYC Motor Vehicle Collisions dataset. The two non-trivial comparable structures implemented from scratch are the KD-tree and Quadtree; Linear Scan is retained as the correctness oracle and baseline.

1 Extended and Refined Proposal

Problem. NYC collision data contains millions of records, but raw CSV search does not make spatial questions easy. GeoPulse asks: how much faster do spatial data structures answer location-based collision queries than a brute force scan?

Motivation. Transportation analysts, students, and residents may want to ask questions such as “what crash is closest to this location?” or “how many crashes happened within half a mile?” A spatial index can reduce repeated query cost after one build step.

Features implemented. GeoPulse loads and validates CSV data, stores accepted collisions in memory, builds three indexes, runs nearest-neighbor, radius, and rectangular range queries, verifies KD-tree and Quadtree answers against Linear Scan, and prints a benchmark report. Recovery mode writes a checkpoint and accepted-record cache so interrupted CSV loads can resume.

Data. The public dataset is NYC Open Data’s Motor Vehicle Collisions Crashes table, export id `h9gi-nx95`. The uploaded snapshot contains 2,246,476 CSV rows and 29 columns. GeoPulse accepts 2,005,745 rows after strict numeric latitude/longitude validation; 240,625 rows are missing coordinates and 106 contain non-empty invalid coordinates.

Tools. The implementation uses C++14, CMake, Catch2, Git/GitHub, Git LFS-ready data layout, VS Code and CLion project settings, optional clang-format, clang-tidy, and cpplint. No library KD-tree or Quadtree is used.

Algorithms implemented. Linear Scan checks every point and is used as the oracle. KD-tree recursively chooses medians on alternating x/y axes. Quadtree recursively partitions the bounding rectangle into four quadrants.

Additional structures. Shared records/points vectors avoid duplicate metadata in tree nodes. Bounding boxes support pruning. Result normalization sorts ids before set comparison.

Role distribution. This was completed as a solo project. I performed proposal refinement, architecture, data ingestion, algorithms, testing, benchmarking, documentation, and packaging.

2 Architecture and Design Decisions

The code follows single-responsibility modules: `core` owns geometry and constants; `data` owns CSV parsing, validation, loading, and recovery; `index` owns the three comparable spatial structures; `query` owns coordinate-facing query helpers; `benchmark` owns timing and correctness comparison; `app/ui` owns output and command dispatch. `main.cpp` only creates `CliApplication` and delegates.

The major post-proposal refinement was adding a stricter loader. The proposal counted rows with non-empty coordinates; the final code accepts only finite, numeric, legal latitude/longitude values. I also added quoted-newline CSV support after discovering that a physical-line parser can miscount this dataset. Recovery was designed with an accepted-record cache because a byte offset alone cannot restore records that were already parsed before a restart.

Structure	Representation	Reason
Linear Scan	Non-owning pointer to all points	Simple oracle; every result is exact.
KD-tree	Unique-pointer binary nodes with axis field	Median partitions improve average repeated query time.
Quadtree	Unique-pointer quadrant nodes with capacity/depth limits	Spatial boxes give intuitive radius/range pruning.

3 Complexity Analysis

Let N be indexed points and K be returned matches. Worst cases remain linear when data is adversarial, duplicate-heavy, or the query covers most of the map.

Command / method	Worst-case	Explanation
<code>--data / DataLoader::load</code>	$O(N)$	Streams each CSV row once.
<code>LinearScan::build</code>	$O(1)$	Stores a reference to the point vector.
<code>LinearScan::nearest</code>	$O(N)$	Checks every accepted point.
<code>LinearScan::radiusQuery</code>	$O(N)$	Checks every distance.
<code>LinearScan::rangeQuery</code>	$O(N)$	Checks every bounding-box containment.
<code>KDTree::build</code>	$O(N \log N)$ avg, $O(N^2)$ worst	Median selection per level.
<code>KDTree::nearest</code>	$O(N)$	Branch pruning may fail.
<code>KDTree::radiusQuery</code>	$O(N + K)$	Large circles can touch all branches.
<code>KDTree::rangeQuery</code>	$O(N + K)$	Large boxes can touch all branches.
<code>QuadTree::build</code>	$O(N \log N)$ avg, $O(ND)$ worst	D is max depth.
<code>QuadTree::nearest</code>	$O(N)$	Box pruning may fail.
<code>QuadTree::radiusQuery</code>	$O(N + K)$	Large circles can touch all nodes.
<code>QuadTree::rangeQuery</code>	$O(N + K)$	Large boxes can touch all nodes.
<code>BenchmarkRunner::run</code>	$O(N \log N + QN)$	Includes baseline correctness checks.

4 Testing, Tooling, and Output

Catch2 tests cover argument validation, exact public sample output, quoted CSV fields, data-loader row counts, recovery cache reuse, geometry, empty-index behavior, negative radius behavior, deterministic nearest ties, KD-tree and Quadtree correctness against Linear Scan, benchmark correctness flags, and a style guard for student files outside Catch2. The public sample output is deterministic:

Dataset summary

```
total rows: 8
accepted coordinate rows: 6
excluded rows: 2
missing-coordinate rows: 1
invalid-coordinate rows: 1
```

Local quality tooling includes CMake targets **Main** and **Tests**, configure-time sample data copying, GitHub settings, branch-safe hooks, Git LFS attributes for snapshots, VS Code and CLion settings, `.clang-format` for Allman braces and 80 columns, and `.clang-tidy` with analyzer, bugprone, performance, readability, and modernize checks. Disabled tidy checks are documented as style conflicts: trailing return types are not used in this beginner-friendly C++14 code, auto is not forced, magic numbers are handled by shared constants and tests, and identifier-length warnings conflict with common loop/index names.

5 Reflection and Lessons Learned

Working solo meant all architecture, implementation, testing, and reporting had to stay highly organized. The most important challenge was not writing the tree code itself; it was making all structures comparable under the same distance model and proving correctness against a baseline. Another challenge was the

CSV format: quoted newlines made it clear that robust data ingestion matters before benchmarking can be trusted.

If I restarted the project, I would implement the public facade, exact sample outputs, and style guard even earlier. I learned how to separate domain records from lightweight index points, why recovery needs persisted accepted records, how KD-tree and Quadtree pruning differ, and why benchmarks must verify answers before interpreting speed.

6 References

NYC Open Data. *Motor Vehicle Collisions - Crashes*, dataset id `h9gi-nx95`.

University of Florida COP3530 Project 2 assignment description and rubric.

Catch2 unit testing framework documentation.